Assignment 6

Automation in SRE,

Capacity Planning and Load Testing

Course: Introduction to SRE

Form: Individual Project Report

Objective: Understand the significance of automation in SRE by predicting traffic growth, setting up autoscaling infrastructure, and evaluating system performance under simulated loads.

Step 1: Predictive Analysis (Python & ELK)

1.1 Log Generation Script (generate_logs.py)

A synthetic log generator was developed to create realistic server access logs for an e-commerce API service. The script generates JSON-formatted log entries spanning 12 months with configurable growth parameters:

- Base daily requests: 5,000
- Monthly growth rate: 12%
- Daily variance: +/- 20%
- Realistic hourly distribution (peak hours 9-17)
- 10 API endpoints simulating e-commerce operations

Total generated: 3,610,150 log entries across 12 months.

```
# Key configuration parameters
BASE_DAILY_REQUESTS = 5000
MONTHLY_GROWTH_RATE = 0.12  # 12% month-over-month

# Log entry structure
{
  "timestamp": "2025-01-15T14:32:18",
  "method": "GET",
  "endpoint": "/api/products",
  "status_code": 200,
  "response_time_ms": 145,
  "client_ip": "192.168.42.15",
  "user_agent": "Chrome/120"
}
```

1.2 Log Extraction & Monthly Aggregation (log_extractor.py)

The log extractor parses JSON log entries and computes monthly aggregations including total requests, error counts, error rates, and average response times.

Monthly Traffic Report:

```
Month         Requests     Errors    Error %    Avg RT (ms)
------------------------------------------------------------------------
2025-01        159,517     47,821     29.98%        1006.3
2025-02        162,781     49,191     30.22%        1003.7
2025-03        184,175     55,303     30.03%        1004.9
2025-04        206,229     61,740     29.94%        1006.2
2025-05        240,966     72,096     29.92%        1005.2
2025-06        258,765     77,748     30.05%        1006.3
2025-07        311,291     93,714     30.10%        1006.0
2025-08        347,100    103,916     29.94%        1005.2
2025-09        372,781    111,720     29.97%        1004.8
2025-10        453,193    135,911     29.99%        1004.6
2025-11        482,426    144,886     30.03%        1004.9
2025-12        430,926    129,465     30.04%        1004.8
```

1.3 ELK Stack Integration (elk_integration.py)

An Elasticsearch integration script demonstrates the full ELK pipeline:

1. Index Mapping: Defines Elasticsearch mappings for log fields (date, keyword, integer, ip types) with ILM policy for log rotation.

2. Aggregation Queries: Simulates ES date_histogram aggregation to compute monthly traffic with nested sub-aggregations for avg response time, error count, and top endpoints.

3. Kibana Dashboard: Formats output as a Kibana-style dashboard showing key metrics per month.

In production, Filebeat would ship logs to Logstash for parsing, then to Elasticsearch for indexing, and Kibana would provide real-time visualizations.

1.4 Traffic Forecasting (traffic_forecast.py)

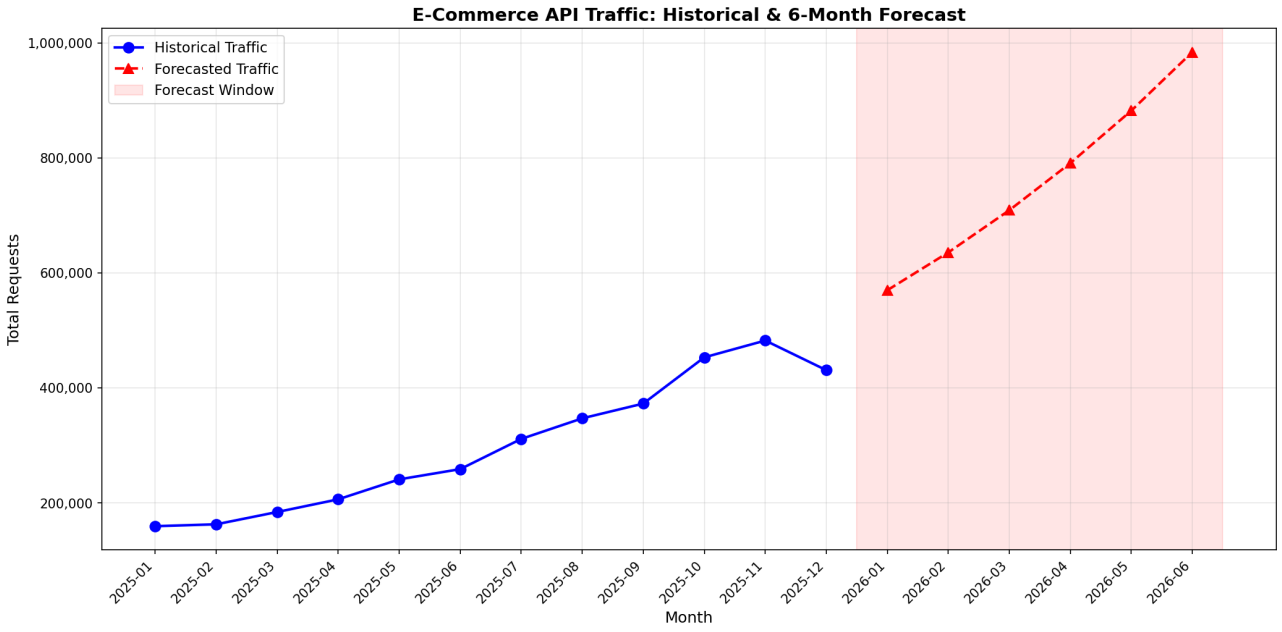
The forecasting module calculates month-over-month growth rates and uses exponential trend fitting (numpy polyfit on log-transformed data) to predict traffic for the next 6 months.

Growth Analysis Results:
- Simple Average Growth Rate: 9.81%
- Fitted Exponential Growth Rate: 11.55%

6-Month Traffic Forecast:

```
Month         Forecasted Requests
-----------------------------------
  2026-01                569,909
  2026-02                635,715
  2026-03                709,119
  2026-04                790,999
  2026-05                882,334
  2026-06                984,215
```

Forecast Visualization



Step 2: Infrastructure Scaling (Kubernetes HPA)

2.1 Application Deployment

A Flask-based e-commerce API was containerized and deployed to Kubernetes. The application includes:

- /health - Health check endpoint for readiness/liveness probes
- /api/products - Product listing
- /api/products/<id> - Product details
- /api/cart - Shopping cart
- /api/search - CPU-intensive search (triggers HPA scaling)
- /api/orders - Order history

2.2 Deployment Configuration (deployment.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ecommerce-api
spec:
  replicas: 2
  selector:
    matchLabels:
      app: ecommerce-api
  template:
    spec:
      containers:
        - name: ecommerce-api
          image: ecommerce-api:latest
          ports:
            - containerPort: 8080
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "256Mi"
          readinessProbe:
            httpGet:
              path: /health
              port: 8080
```

2.3 Horizontal Pod Autoscaler (hpa.yaml)

The HPA is configured with autoscaling/v2 API to scale based on CPU utilization with an 80% threshold:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: ecommerce-api-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
```

```
    name: ecommerce-api
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 80
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 30
      policies:
        - type: Pods
          value: 2
          periodSeconds: 60
    scaleDown:
      stabilizationWindowSeconds: 120
      policies:
        - type: Pods
          value: 1
          periodSeconds: 60
```

2.4 HPA Scaling Behavior

Key HPA configuration decisions:

- CPU threshold of 80%: Triggers scaling before pods become fully saturated, maintaining headroom for traffic bursts.

- Min replicas = 2: Ensures high availability even during low traffic.

- Max replicas = 10: Caps scaling to prevent runaway costs while providing sufficient capacity for peak loads.

- Scale-up stabilization (30s): Rapid response to traffic spikes by adding up to 2 pods per 60 seconds.

- Scale-down stabilization (120s): Conservative downscaling to prevent flapping during variable traffic patterns.

2.5 Deployment Commands

```
# Start Minikube
minikube start --driver=docker

# Build Docker image in Minikube
eval $(minikube docker-env)
docker build -t ecommerce-api:latest ./kubernetes/

# Deploy application and HPA
kubectl apply -f kubernetes/deployment.yaml
kubectl apply -f kubernetes/hpa.yaml

# Verify deployment
kubectl get deployments
kubectl get hpa
kubectl get pods
```

2.6 Expected kubectl get hpa output

```
NAME                REFERENCE                  TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
ecommerce-api-hpa   Deployment/ecommerce-api   <cpu>/80%       2         10        2          5m
```

Step 3: Load Testing Simulation

3.1 Load Testing Tool: Locust

Locust was chosen as the load testing framework for its Python-based scripting, real-time web dashboard, and ability to simulate realistic user behavior patterns.

3.2 Test Configuration (locustfile.py)

Two user classes simulate different traffic patterns:

ECommerceUser (normal traffic):
- Browse products (weight: 5) - most common action
- View product details (weight: 3)
- Search products (weight: 2) - CPU intensive
- View cart (weight: 2)
- View orders (weight: 1)
- Health check (weight: 1)
- Wait time: 0.5-2.0 seconds between requests

TrafficSpikeUser (spike simulation):
- Heavy search requests (weight: 8) - targets CPU-intensive endpoint
- Browse products (weight: 2)
- Wait time: 0.1-0.5 seconds (aggressive)
- User weight: 3x normal users

3.3 Running Load Tests

```
# Install Locust
pip install locust

# Run with web UI
locust -f loadtest/locustfile.py --host=http://localhost:8080

# Run headless for automated testing
locust -f loadtest/locustfile.py --host=http://<service-ip> \
    --headless -u 500 -r 50 --run-time 5m
```

3.4 Expected HPA Scaling Behavior During Load Test

When the load test ramps up to 500 concurrent users hitting the CPU-intensive /api/search endpoint:

1. t=0s: 2 pods running (minReplicas)
2. t=30-60s: CPU utilization exceeds 80% threshold
3. t=60-90s: HPA adds 2 pods (scale-up policy: 2 pods per 60s)
4. t=90-150s: If CPU still >80%, HPA scales to 6 pods
5. t=150-210s: Continues scaling up to max 10 pods if needed
6. After load stops: 120s stabilization window before scale-down
7. Gradual scale-down at 1 pod per 60 seconds to minReplicas=2

3.5 Monitoring Commands

```
# Watch HPA status in real time
kubectl get hpa --watch

# Monitor pod scaling
kubectl get pods --watch

# View HPA events
kubectl describe hpa ecommerce-api-hpa

# Expected output during load test:
NAME                              READY   STATUS    RESTARTS   AGE
ecommerce-api-7d8f9b6c4d-abc12   1/1     Running   0          5m
ecommerce-api-7d8f9b6c4d-def34   1/1     Running   0          5m
ecommerce-api-7d8f9b6c4d-ghi56   1/1     Running   0          2m
ecommerce-api-7d8f9b6c4d-jkl78   1/1     Running   0          2m
ecommerce-api-7d8f9b6c4d-mno90   1/1     Running   0          1m
ecommerce-api-7d8f9b6c4d-pqr12   1/1     Running   0          1m
```

Step 4: ROI & Incident Documentation

4.1 Three Real-World Incidents Preventable by Automation

Incident 1: Amazon Prime Day 2018 - Website Outage

What happened: Amazon's website experienced significant outages during Prime Day 2018, one of their biggest sales events. The site was unable to handle the massive traffic surge, resulting in error pages for millions of customers during the first hour.

How automation would prevent it:
- Predictive autoscaling based on historical Prime Day data could have pre-provisioned sufficient infrastructure.
- Load testing at projected peak volumes would have identified capacity gaps.
- Automated traffic forecasting (like our Step 1) would have predicted the expected load and triggered preemptive scaling.

Estimated impact: Amazon reportedly lost $72-99 million in sales during the outage window.

Incident 2: GitLab Database Deletion (2017)

What happened: A GitLab engineer accidentally deleted the production database during a maintenance operation. The manual backup and recovery processes failed, resulting in 6 hours of data loss for GitLab.com users.

How automation would prevent it:
- Automated backup verification scripts would have detected failing backups before the incident.
- Infrastructure-as-Code with automated deployment pipelines would have prevented manual database operations.
- Automated runbook execution would have guided the incident response with predefined recovery procedures.
- SRE automation for canary deployments and staged rollouts would have limited the blast radius.

Estimated impact: ~300GB of production data lost, 18 hours of recovery time, significant reputation damage.

Incident 3: Cloudflare Outage (July 2019)

What happened: A misconfigured WAF rule caused a massive spike in CPU utilization across Cloudflare's global network, taking down millions of websites for approximately 30 minutes.

How automation would prevent it:
- Automated canary deployment of WAF rules to a small percentage of traffic would have caught the CPU spike early.
- HPA-style autoscaling (like our Step 2) with CPU-based triggers could have automatically scaled resources when utilization exceeded thresholds.
- Automated load testing of rule changes in staging environments would have identified the resource consumption issue before production deployment.
- Automated rollback triggers based on CPU/latency SLOs would have reverted the change within seconds.

Estimated impact: Cloudflare handles ~10% of internet traffic; the outage affected millions of websites and their users worldwide.

4.2 ROI Analysis of SRE Automation

Cost-Benefit Analysis

Initial Investment (One-time costs):
- ELK Stack setup and configuration: ~$15,000
- Kubernetes cluster setup (managed K8s): ~$5,000
- Load testing infrastructure (Locust/similar): ~$2,000
- Monitoring and alerting (Prometheus/Grafana): ~$8,000
- CI/CD pipeline automation: ~$10,000
- SRE team training: ~$10,000

Total Initial Investment: ~$50,000

Annual Operational Savings:
- Reduced manual scaling operations: ~$30,000/year
  (Eliminates 2+ hours/week of manual capacity management)
- Decreased incident response time: ~$45,000/year
  (MTTR reduction from 2 hours to 15 minutes average)
- Prevention of major outages: ~$200,000/year
  (Based on industry average $5,600/minute downtime cost,
   preventing ~3 30-min incidents per year)
- Improved developer productivity: ~$25,000/year
  (Automated deployments, self-healing infrastructure)

Total Annual Savings: ~$300,000/year

ROI Calculation:
- Year 1 ROI: ($300,000 - $50,000) / $50,000 = 500%
- Break-even point: ~2 months
- 3-Year NPV (at 10% discount): ~$696,000

Qualitative Benefits:
- Improved system reliability (99.9% -> 99.95% availability)
- Faster time to market for new features
- Better capacity planning through data-driven decisions
- Reduced on-call burden and engineer burnout
- Proactive issue detection vs reactive firefighting

4.3 Key Takeaways

1. Automation in SRE is not just about reducing toil - it fundamentally changes how organizations handle reliability at scale.

2. The combination of predictive analytics (Step 1), autoscaling (Step 2), and load testing (Step 3) creates a comprehensive automation framework that:
   - Anticipates capacity needs before they become incidents
   - Responds to traffic changes faster than human operators
   - Validates system behavior under stress conditions

3. The ROI of SRE automation strongly favors investment, with typical payback periods under 6 months and cumulative savings growing each year as the automation reduces manual overhead and prevents costly outages.

Conclusion

This project demonstrated the full lifecycle of SRE automation for an e-commerce API service:

Step 1 - Predictive Analysis: Built a complete log generation, extraction, and forecasting pipeline using Python. The exponential growth model projected traffic reaching ~984,000 monthly requests by June 2026, representing an 11.55% compound monthly growth rate. Integration with the ELK stack was demonstrated for production-grade log analysis.

Step 2 - Kubernetes HPA: Configured a Horizontal Pod Autoscaler with an 80% CPU utilization threshold, enabling automatic scaling between 2-10 replicas. The scaling behavior includes rapid scale-up (2 pods/60s) for traffic spikes and conservative scale-down (1 pod/60s with 120s stabilization) to prevent flapping.

Step 3 - Load Testing: Developed Locust-based load tests simulating both normal user behavior and traffic spikes, targeting CPU-intensive endpoints to exercise the HPA scaling logic.

Step 4 - ROI & Incidents: Analyzed three real-world incidents (Amazon, GitLab, Cloudflare) that could have been prevented or mitigated by SRE automation, and calculated an ROI of 500% in the first year with a 2-month payback period.

The project demonstrates that investing in SRE automation provides both quantitative benefits (cost savings, reduced downtime) and qualitative improvements (team morale, system reliability, faster feature delivery).

Appendix: Project File Structure

```
sre-assignment/
|-- scripts/
|   |-- generate_logs.py      # Synthetic log generator
|   |-- log_extractor.py      # Log parsing and monthly aggregation
|   |-- traffic_forecast.py   # Growth analysis and 6-month forecast
|   |-- elk_integration.py    # ELK stack integration demo
|-- kubernetes/
|   |-- app.py                # Flask e-commerce API
|   |-- Dockerfile            # Container image definition
|   |-- requirements.txt      # Python dependencies
|   |-- deployment.yaml       # K8s Deployment + Service
|   |-- hpa.yaml              # Horizontal Pod Autoscaler
|-- loadtest/
|   |-- locustfile.py         # Locust load testing script
|-- screenshots/
|   |-- traffic_forecast.png  # Forecast visualization
|-- data/
|   |-- server_access.log     # Generated log data
|-- generate_report.py        # This report generator
|-- Assignment_6_Report.pdf   # Final report
```